

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_87c49086-be4\agent-a459bbde53c2304d4.jsonl`
- SHA-256: `5061afe7ce32963096a4c5647243b5c2650ab9adb77820b2a2d36c046a0b58e7`
- Source modified: `2026-06-18T08:13:07+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_87c49086-be4`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T08:05:20.104Z)

You implement ONE behavior-preserving refactor and open a PR for OWNER REVIEW (do NOT merge). You are in your OWN isolated git worktree of the badminton-highlight-indexer repo on THIS machine.
The Python env has pytest/ruff/mypy/cv2/torch/numpy ? the full offline suite RUNS here (ignore any CLAUDE.md note saying it can't).

SETUP FIRST, exactly:

```
git fetch origin
git checkout -B <BRANCH> origin/master      # base MUST be origin/master, NOT the current HEAD
then verify: `git log --oneline -1` is origin/master's HEAD and `git branch --show-current` == <BRANCH>.
```

RULES (pure refactor ? quality must NOT regress, behavior must NOT change):

- Behavior-PRESERVING moves/extractions ONLY. Do NOT change logic, branch conditions, log strings/levels, return types, ordering, thresholds, or constants. It is cut-paste into helpers.
- Edit ONLY the files named in the task. Stage EXPLICIT paths (`git add <file>`), NEVER -A./-u.
- Match the surrounding code style and naming.

GATE (binding ? run all four; capture pass/skip counts + coverage %):

```
python run_all_tests.py
python -m ruff check .
python -m mypy backend training
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70
```

ONLY IF ALL FOUR ARE GREEN:

```
git add <explicit files>
git commit -m "<conventional commit msg>"    # final line: Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
git push -u origin <BRANCH>
gh pr create --base master --head <BRANCH> --title "<title>" --body "<concise body: what/why, behavior-preserving argument, gate results>"    # NO merge
IF ANY GATE IS RED: do NOT push or open a PR. Fix if it's a trivial slip you introduced;
otherwise
report status='gate-failed' with the failing output and your diff so far. Never weaken a test or lower the coverage floor to go green.
```

Return the structured object with: slug, branch, status ('pr-opened'|'gate-failed'|'error'), pr_url (""+reason if none), diffstat, suite (e.g. '851 passed, 7 skipped'), ruff, mypy, coverage, behavior_note (why it's a pure move), caveat.

=== YOUR TASK: db-init-extract (branch refactor/db-init-helpers) ===

FILE: backend/infrastructure/database.py ? method _init_db (the long migration ladder).

Extract, as a PURE intra-class move: the three base-table CREATES (videos/play_segments/events) into `\_create\_base\_tables(self, cursor)`, and each `if version < N:` block body VERBATIM into `\_migrate\_to\_vN(self, cursor)` (one per N=1..5). `\_init\_db` keeps the SAME single `with self.\_connect()` block: call `\_create\_base\_tables`, read `version = cursor.execute("PRAGMA user\_version").fetchone()[0]`, then call each `\_migrate\_to\_vN` guarded by the SAME `if version < N:` check, in order. Do NOT change any SQL text, the PRAGMA bumps, the `\_add\_column\_idempotent` calls, cursor/commit semantics, or order ? all DDL still runs on the one cursor in the one connection block. TESTS that must stay green (besides the full suite): tests/test_database.py (incl. test_migration_idempotent_on_interrupted_upgrade, test_migration_v3_db_upgrades_to_v5_preserving_rows, test_schema_version_is_current), tests/test_async_jobs.py, tests/test_job_worker_loop.py.

You are running in an isolated git worktree at C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_87c49086-be4-1 (a separate working copy of the repo). Changes you make here do NOT affect the main working directory (C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\refactor-base) or other agents. Work normally ? the worktree will be cleaned up automatically if you made no changes, or preserved for review if you did.

2. assistant (2026-06-18T08:05:23.086Z)

I'll start by setting up the worktree branch from origin/master, then examine the target file.

3. user (2026-06-18T08:05:27.019Z)

```
branch 'refactor/db-init-helpers' set up to track 'origin/master'.
Switched to a new branch 'refactor/db-init-helpers'
---HEAD---
a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
---BRANCH---
refactor/db-init-helpers
```

4. assistant (2026-06-18T08:05:28.892Z)

Branch is correctly set up on origin/master's HEAD (a010667). Now let me read the target file.

5. user (2026-06-18T08:05:30.593Z)

```
1      import sqlite3
2      import json
3      import logging
4      from contextlib import contextmanager
5      from typing import List, Dict, Any, Optional, Tuple
6
7      logger = logging.getLogger(__name__)
8
9      class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
16             """Transaction scope that also CLOSES the connection.
17
18             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19             never closes, so every call leaked its connection to GC (lingering file
20             handles on the WAL sidecars under Windows). Internal methods use this.
21             """
22             conn = self._get_connection()
```

```

23         try:
24             with conn:
25                 yield conn
26         finally:
27             conn.close()
28
29     def _get_connection(self):
30         # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31         # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
32         conn = sqlite3.connect(self.db_path, timeout=30.0)
33         conn.row_factory = sqlite3.Row
34         # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
35         conn.execute("PRAGMA foreign_keys = ON")
36         # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
37         conn.execute("PRAGMA busy_timeout = 30000")
38         try:
39             conn.execute("PRAGMA journal_mode = WAL")
40         except sqlite3.OperationalError:
41             pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
42         return conn
43
44     @staticmethod
45     def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
46         """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runably.
47
48         A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn``
49         transaction, so an interrupted v3/v4 block can leave the column added while
50         ``user_version`` stays un-bumped ? and the next open would re-run the same
51         ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
52         future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
Swallow
53         ONLY that specific error so the ladder stays crash-safe, matching the re-
runnable
54         ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
55         """
56         try:
57             cursor.execute(ddl)
58         except sqlite3.OperationalError as exc:
59             if "duplicate column name" not in str(exc).lower():
60                 raise
61
62     def _init_db(self):
63         """Initializes tables for videos, play_segments, and events."""
64         with self._connect() as conn:
65             cursor = conn.cursor()
66
67             # Videos Table
68             cursor.execute("""
69                 CREATE TABLE IF NOT EXISTS videos (
70                     id TEXT PRIMARY KEY,
71                     filepath TEXT NOT NULL,
72                     processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
73                     status TEXT NOT NULL,
74                     validation_results TEXT
75                 )
76             """)
77
78             # Play Segments Table (Extensible metadata JSON)
79             cursor.execute("""
80                 CREATE TABLE IF NOT EXISTS play_segments (
81                     id INTEGER PRIMARY KEY AUTOINCREMENT,
82                     video_id TEXT NOT NULL,
83                     start_time REAL NOT NULL,
84                     end_time REAL NOT NULL,

```

```

85         duration REAL NOT NULL,
86         server TEXT,
87         receiver TEXT,
88         metadata TEXT,
89         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
90     )
91 """)
92
93 # Events Table
94 cursor.execute("""
95     CREATE TABLE IF NOT EXISTS events (
96         id INTEGER PRIMARY KEY AUTOINCREMENT,
97         segment_id INTEGER NOT NULL,
98         timestamp REAL NOT NULL,
99         type TEXT NOT NULL,
100        player TEXT,
101        details TEXT,
102        FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
103    )
104 """)
105
106 # Versioned migrations live here. PRAGMA user_version is the schema
107 # contract ? bump it for every change and add an ordered `if version < N`
108 # block below. Tests assert the expected version, so accidental drift fails
CI.
109 version = cursor.execute("PRAGMA user_version").fetchone()[0]
110
111 if version < 1:
112     # v1: raw candidate detections (pre-confirmation), detector-agnostic.
113     # Common fields are columns; detector-specific metrics go in `stats`
JSON,
114     # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
115     cursor.execute("""
116         CREATE TABLE IF NOT EXISTS candidate_segments (
117             id INTEGER PRIMARY KEY AUTOINCREMENT,
118             video_id TEXT NOT NULL,
119             detector TEXT NOT NULL,
120             chunk_index INTEGER,
121             start_time REAL NOT NULL,
122             end_time REAL NOT NULL,
123             duration REAL NOT NULL,
124             confidence REAL,
125             stats TEXT,
126             detection_params TEXT,
127             confirmed_segment_id INTEGER,
128             human_label TEXT,
129             notes TEXT,
130             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
131             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
132             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
133         )
134     """)
135     cursor.execute("""
136         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
137         ON candidate_segments(video_id, detector)
138     """)
139     cursor.execute("PRAGMA user_version = 1")
140
141 if version < 2:
142     # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
143     # /api/process request. `status` is the EXECUTION state of the job

```

```

144         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
145         # that failed validation) lives inside `result` JSON as
result["status"].
146         # `result` is the full sync-era response dict (incl. report paths), so
the
147         # observability report is the job's artifact, per the plan.
148         cursor.execute("""
149             CREATE TABLE IF NOT EXISTS jobs (
150                 id TEXT PRIMARY KEY,
151                 video_path TEXT NOT NULL,
152                 video_id TEXT,
153                 segmenter TEXT,
154                 player_pool TEXT,
155                 max_frames INTEGER,
156                 status TEXT NOT NULL DEFAULT 'queued',
157                 progress TEXT,
158                 error TEXT,
159                 result TEXT,
160                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
161                 started_at TIMESTAMP,
162                 finished_at TIMESTAMP
163             )
164         """)
165         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
166         cursor.execute("PRAGMA user_version = 2")
167
168     if version < 3:
169         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
170         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
171         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
172         # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
173         # the coordinator. ALTER (not recreate) so existing v2 job rows survive;
the
174         # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-
run them.
175         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy
TEXT")
176         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN
next_poll_at TIMESTAMP")
177         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
178         cursor.execute("PRAGMA user_version = 3")
179
180     if version < 4:
181         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
NULLABLE
182         # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
183         # path can scope rows to an owner. **NULL = local install = byte-
identical to
184         # today** ? every existing row stays NULL and every existing query
(which never
185         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive;
186         # the ADD COLUMNS are applied idempotently so an interrupted upgrade can
re-run them.
187         # Additive ONLY: no served-behavior change rides on this slice.
188         self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN
user_id TEXT")
189         self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD

```

```

COLUMN user_id TEXT")
190 # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
path free
191 # of index churn while making the eventual per-user lookups cheap.
192 cursor.execute(
193     "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
194     "WHERE user_id IS NOT NULL")
195 cursor.execute(
196     "CREATE INDEX IF NOT EXISTS idx_candidates_user "
197     "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
198 cursor.execute("PRAGMA user_version = 4")
199
200 if version < 5:
201     # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
202     # (user_id,
203     # version): the resolved per-user `fusion_config` overlay + an INLINE
204     # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
205     # evidence
206     # and provenance that earned it. `is_active` pins the one row the
207     # serving bridge
208     # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
209     # only persists/loads it ? NOTHING reads it on the served path yet.
210     #
211     # user_id          ? owner of the model (NULL allowed = the local
212     #
213     # version          ? monotonic per-user model version
214     # baseline_version ? the shared baseline this was fit against
215     #
216     # feature_schema_hash ? fingerprint of the feature set; MINOR-vs-
217     #
218     # fusion_config     ? JSON per-user FusionConfig overlay (cue_flags
219     #
220     # classifier_json   ? JSON LogisticRegression.to_dict() (None =
221     #
222     # promotion_evidence ? JSON per_user_gate PromotionDecision (why it
223     #
224     # n_videos_at_fit   ? held-out-user corpus size when fit (min_n
225     #
226     # is_active         ? the one resolved row for this user (partial-
227     #
228     # unique below)
229     cursor.execute("""
230         CREATE TABLE IF NOT EXISTS user_models (
231             id INTEGER PRIMARY KEY AUTOINCREMENT,
232             user_id TEXT,
233             version INTEGER NOT NULL DEFAULT 1,
234             baseline_version TEXT,
235             feature_schema_hash TEXT,
236             fusion_config TEXT,
237             classifier_json TEXT,
238             promotion_evidence TEXT,
239             n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
240             is_active INTEGER NOT NULL DEFAULT 0,
241             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
242         )
243     """)
244     # One model version per user (NULLs compare distinct in SQLite, so the
245     # local
246     # install can still carry multiple versioned rows; the active-row guard
247     # below
248     # is what enforces a single served model).
249     cursor.execute("""
250         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
251         ON user_models(user_id, version)
252     """)

```

```

239         # At most ONE active model per user: a partial-unique index over the
active rows.
240         # (SQLite treats NULL user_id rows as distinct, so the local install's
single
241         # active row is still guarded ? there is one local user.)
242         cursor.execute("""
243             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
244             ON user_models(user_id) WHERE is_active = 1
245         """)
246         cursor.execute("PRAGMA user_version = 5")
247         # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6
248
249         conn.commit()
250
251     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
252         """Register or update a video row WITHOUT touching its child segments.
253
254         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
255         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
256         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
257         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
258         row in place; clearing segments is now an explicit, deliberate operation
259         (clear_video_data / prune_segments).
260         """
261         try:
262             with self._connect() as conn:
263                 conn.cursor().execute(
264                     "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
265                     "ON CONFLICT(id) DO UPDATE SET "
266                     "filepath=excluded.filepath, status=excluded.status, "
267                     "validation_results=excluded.validation_results",
268                     (video_id, filepath, status, json.dumps(validation_results))
269                 )
270                 conn.commit()
271             return True
272         except Exception as e:
273             logger.error(f"Failed to add video: {e}")
274             return False
275
276     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
277         """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
278
279         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
280         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
281         """
282         with self._connect() as conn:
283             cur = conn.cursor()
284             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
285                             (video_id,)).fetchone()[0]
286             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
287                             (video_id,)).fetchone()[0]
288             return int(p), int(c)
289
290     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
291                       play_after: Optional[int] = None, cand_upto: Optional[int] =
None,

```

```

292             cand_after: Optional[int] = None) -> None:
293         """Delete play/candidate segments by id range (events cascade from
play_segments).
294
295         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
296         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
297         and keep the OLD ones when the run failed/produced nothing (`*_after`).
298         """
299         with self._connect() as conn:
300             cur = conn.cursor()
301             if play_upto is not None:
302                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
303             if play_after is not None:
304                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
305             if cand_upto is not None:
306                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
307             if cand_after is not None:
308                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
309             conn.commit()
310
311         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
312             try:
313                 with self._connect() as conn:
314                     cursor = conn.cursor()
315                     cursor.execute(
316                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
317                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
318                     )
319                     conn.commit()
320                     return cursor.lastrowid
321             except Exception as e:
322                 logger.error(f"Failed to add play segment: {e}")
323                 return None
324
325         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
326             try:
327                 with self._connect() as conn:
328                     conn.cursor().execute(
329                         "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
330                         (segment_id, timestamp, event_type, player, json.dumps(details or
{})))
331                     )
332                     conn.commit()
333                     return True
334             except Exception as e:
335                 logger.error(f"Failed to add event: {e}")
336                 return False
337
338         def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
339                                 chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
340                                 stats: Optional[Dict[str, Any]] = None,
341                                 detection_params: Optional[Dict[str, Any]] = None) ->

```

```

Optional[int]:
342     """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
343     detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
344     try:
345         with self._connect() as conn:
346             cursor = conn.cursor()
347             cursor.execute(
348                 """INSERT INTO candidate_segments
349                     (video_id, detector, chunk_index, start_time, end_time, duration,
350                      confidence, stats, detection_params)
351                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
352                 (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
353                  confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
354             )
355             conn.commit()
356             return cursor.lastrowid
357     except Exception as e:
358         logger.error(f"Failed to add candidate segment: {e}")
359         return None
360
361     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
362         """Record that a candidate detection was confirmed as a given play_segment."""
363         try:
364             with self._connect() as conn:
365                 conn.cursor().execute(
366                     "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
367                     (segment_id, candidate_id)
368                 )
369                 conn.commit()
370                 return True
371         except Exception as e:
372             logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
373             return False
374
375     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
376                                only_unlabeled: bool = False) -> List[Dict[str, Any]]:
377         """Fetch candidate detections for the annotation / fine-tuning workflow.
378         `stats` and `detection_params` are deserialized from JSON."""
379         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
380         params: List[Any] = [video_id]
381
382         if detector:
383             query += " AND detector = ?"
384             params.append(detector)
385         if only_unlabeled:
386             query += " AND human_label IS NULL"
387
388         query += " ORDER BY start_time"
389
390         candidates = []
391         with self._connect() as conn:
392             rows = conn.cursor().execute(query, params).fetchall()
393             for row in rows:
394                 d = dict(row)
395                 d["stats"] = json.loads(d["stats"] or "{}")
396                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
397                 candidates.append(d)
398         return candidates
399
400     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:

```

```

401         with self._connect() as conn:
402             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
403             if row:
404                 d = dict(row)
405                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
406                 return d
407             return None
408
409     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
410         """
411         Dynamically queries play segments based on filters:
412         - duration_min: float
413         - server: string
414         - receiver: string
415         - event_type: string (e.g. smash, foul)
416         """
417         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
418         params = [video_id]
419
420         if filters.get("duration_min"):
421             query += " AND r.duration >= ?"
422             params.append(filters["duration_min"])
423
424         if filters.get("server"):
425             query += " AND r.server = ?"
426             params.append(filters["server"])
427
428         if filters.get("receiver"):
429             query += " AND r.receiver = ?"
430             params.append(filters["receiver"])
431
432         if filters.get("event_type"):
433             # Check if there is an event of a specific type inside this segment
434             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
435             params.append(filters["event_type"])
436
437         segments_list = []
438         with self._connect() as conn:
439             cursor = conn.cursor()
440             rows = cursor.execute(query, params).fetchall()
441
442             for row in rows:
443                 r_dict = dict(row)
444                 r_id = r_dict["id"]
445                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
446
447                 # Fetch associated events
448                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
449                 r_dict["events"] = []
450                 for er in event_rows:
451                     e_dict = dict(er)
452                     e_dict["details"] = json.loads(e_dict["details"] or "{}")
453                     r_dict["events"].append(e_dict)
454
455                 segments_list.append(r_dict)
456
457             return segments_list
458
459     def clear_video_data(self, video_id: str):
460         with self._connect() as conn:
461             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))

```

```

462         conn.commit()
463
464     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
465     # Writers follow the repo convention (swallow + logger.error, return bool); the
466     # worker logs loudly on a failed transition so a job can't silently wedge.
467
468     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
469                   player_pool: Optional[List[str]] = None,
470                   max_frames: Optional[int] = None,
471                   policy: Optional[Dict[str, Any]] = None) -> bool:
472         """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
473         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
474         try:
475             with self._connect() as conn:
476                 conn.cursor().execute(
477                     "INSERT INTO jobs (id, video_path, segmenter, player_pool,
max_frames, status, policy) "
478                     "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
479                     (job_id, video_path, segmenter,
480                      json.dumps(player_pool) if player_pool is not None else None,
481                      max_frames, json.dumps(policy) if policy is not None else None)
482                 )
483                 conn.commit()
484             return True
485         except Exception as e:
486             logger.error(f"Failed to create job {job_id}: {e}")
487             return False
488
489     def mark_job_running(self, job_id: str) -> bool:
490         try:
491             with self._connect() as conn:
492                 conn.cursor().execute(
493                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
494                     "progress='starting' WHERE id = ?", (job_id,))
495                 conn.commit()
496             return True
497         except Exception as e:
498             logger.error(f"Failed to mark job {job_id} running: {e}")
499             return False
500
501     def set_job_progress(self, job_id: str, progress: str) -> bool:
502         try:
503             with self._connect() as conn:
504                 conn.cursor().execute(
505                     "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id))
506                 conn.commit()
507             return True
508         except Exception as e:
509             logger.error(f"Failed to set progress for job {job_id}: {e}")
510             return False
511
512     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
513         try:
514             with self._connect() as conn:
515                 conn.cursor().execute(
516                     "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id))
517                 conn.commit()
518             return True
519         except Exception as e:
520             logger.error(f"Failed to set video_id for job {job_id}: {e}")
521             return False
522
523     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
524         """Job EXECUTED to completion. The pipeline's own verdict (success/failed

```

```

525         validation) is inside `result` ? job status 'done' means 'the worker
finished'. """
526         try:
527             with self._connect() as conn:
528                 conn.cursor().execute(
529                     "UPDATE jobs SET status='done', progress='finished', result=?, "
530                     "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
531                     (json.dumps(result), job_id))
532                 conn.commit()
533             return True
534         except Exception as e:
535             logger.error(f"Failed to mark job {job_id} done: {e}")
536             return False
537
538     def mark_job_failed(self, job_id: str, error: str) -> bool:
539         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
540         try:
541             with self._connect() as conn:
542                 conn.cursor().execute(
543                     "UPDATE jobs SET status='failed', error=?, "
544                     "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id))
545                 conn.commit()
546             return True
547         except Exception as e:
548             logger.error(f"Failed to mark job {job_id} failed: {e}")
549             return False
550
551     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
552         """Fetch one job; `result`/`player_pool` JSON deserialized."""
553         with self._connect() as conn:
554             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
555             if not row:
556                 return None
557             d = dict(row)
558             d["result"] = json.loads(d["result"]) if d["result"] else None
559             d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
None
560             d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
561             return d
562
563     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
-----
564     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
565         """Attach/replace a job's JSON ExecutionPolicy."""
566         try:
567             with self._connect() as conn:
568                 conn.cursor().execute(
569                     "UPDATE jobs SET policy=? WHERE id = ?",
570                     (json.dumps(policy) if policy is not None else None, job_id))
571                 conn.commit()
572             return True
573         except Exception as e:
574             logger.error(f"Failed to set policy for job {job_id}: {e}")
575             return False
576
577     def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
578         """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
579         worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
580         computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
581         try:

```

```

582         with self._connect() as conn:
583             conn.cursor().execute(
584                 "UPDATE jobs SET status='waiting', "
585                 "next_poll_at=datetime('now', ?), "
586                 "progress=COALESCE(?, progress) WHERE id = ?",
587                 (f"{max(0, int(delay_sec))} seconds", progress, job_id))
588             conn.commit()
589         return True
590     except Exception as e:
591         logger.error(f"Failed to set job {job_id} waiting: {e}")
592         return False
593
594     def seconds_until_next_due(self) -> Optional[float]:
595         """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
596         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
597         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
598         try:
599             with self._connect() as conn:
600                 row = conn.cursor().execute(
601                     "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
602                     "    (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
603                     "FROM jobs WHERE status='waiting')).fetchone()
604                 if row is None or row["secs"] is None:
605                     return None
606                 return max(0.0, float(row["secs"]))
607     except Exception as e:
608         logger.error(f"Failed to compute next-due delay: {e}")
609         return None
610
611     def claim_due_job(self) -> Optional[Dict[str, Any]]:
612         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
613         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
614         double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
615         first. Returns the claimed job dict, or None if nothing is runnable."""
616         try:
617             with self._connect() as conn:
618                 cur = conn.cursor()
619                 row = cur.execute(
620                     "SELECT id, status FROM jobs WHERE status='queued' "
621                     "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
622                     "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
623                 if not row:
624                     return None
625                 cur.execute(
626                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
627                     "WHERE id=? AND status=?", (row["id"], row["status"]))
628                 conn.commit()
629                 if cur.rowcount != 1:
630                     return None # lost the race to a concurrent
worker
631                 return self.get_job(row["id"])
632     except Exception as e:
633         logger.error(f"Failed to claim a due job: {e}")
634         return None
635
636     def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:

```

```

637         """Crash recovery: any job still queued/running from a previous process is dead
638         (the executor is in-process). Mark failed so clients see a terminal state, not a
639         hang. Called once at startup. Returns the number of jobs swept."""
640         try:
641             with self._connect() as conn:
642                 cur = conn.cursor()
643                 cur.execute(
644                     "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
645                     "WHERE status IN ('queued', 'running')", (reason,))
646                 conn.commit()
647                 return cur.rowcount
648         except Exception as e:
649             logger.error(f"Failed to sweep stale jobs: {e}")
650             return 0
651
652     # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) -----
653     # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
654     # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
655     # serving.py) resolves them, and wiring that bridge into the production decision
seam
656     # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
657     # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
658     # JSON-serialised dicts; the rest are scalar columns.
659
660     def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
661                           baseline_version: Optional[str] = None,
662                           feature_schema_hash: Optional[str] = None,
663                           fusion_config: Optional[Dict[str, Any]] = None,
664                           classifier_json: Optional[Dict[str, Any]] = None,
665                           promotion_evidence: Optional[Dict[str, Any]] = None,
666                           n_videos_at_fit: int = 0,
667                           is_active: bool = False) -> Optional[int]:
668         """Insert or replace ONE per-user model row keyed by `(user_id, version)`.
669
670         Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
the
671         rowid). When `is_active` is True this row becomes the user's single active
model ?
672         any previously-active row for the SAME user is first cleared, so the
673         `idx_user_models_one_active` partial-unique invariant always holds (one active
model
674         per user). Returns the row id, or None on failure.
675         """
676         try:
677             with self._connect() as conn:
678                 cur = conn.cursor()
679                 if is_active:
680                     # Demote the current active row first (NULL user_id = local: IS NULL
match).
681                     if user_id is None:
682                         cur.execute(
683                             "UPDATE user_models SET is_active = 0 "
684                             "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
685                             (version,))
686                     else:
687                         cur.execute(
688                             "UPDATE user_models SET is_active = 0 "
689                             "WHERE user_id = ? AND is_active = 1 AND version != ?",
690                             (user_id, version))
691                 cur.execute(
692                     """INSERT INTO user_models
693                     (user_id, version, baseline_version, feature_schema_hash,
694                     fusion_config, classifier_json, promotion_evidence,
695                     n_videos_at_fit, is_active)

```

```

696         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
697     ON CONFLICT(user_id, version) DO UPDATE SET
698         baseline_version=excluded.baseline_version,
699         feature_schema_hash=excluded.feature_schema_hash,
700         fusion_config=excluded.fusion_config,
701         classifier_json=excluded.classifier_json,
702         promotion_evidence=excluded.promotion_evidence,
703         n_videos_at_fit=excluded.n_videos_at_fit,
704         is_active=excluded.is_active"",
705     (user_id, version, baseline_version, feature_schema_hash,
706     json.dumps(fusion_config) if fusion_config is not None else None,
707     json.dumps(classifier_json) if classifier_json is not None else
None,
708     json.dumps(promotion_evidence) if promotion_evidence is not None
else None,
709     n_videos_at_fit, 1 if is_active else 0))
710     conn.commit()
711     rid = cur.execute(
712         "SELECT id FROM user_models WHERE "
713         + ("user_id IS NULL" if user_id is None else "user_id = ?")
714         + " AND version = ?",
715         ((version,) if user_id is None else (user_id, version))).fetchone()
716     return int(rid[0]) if rid else None
717 except Exception as e:
718     logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
{e}")
719     return None
720
721 def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
722     """Return the single active per-user model row for ``user_id`` (NULL = local
install),
723     with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
deserialised,
724     or None if the user has no active model. Does not touch the served pipeline."""
725     with self._connect() as conn:
726         if user_id is None:
727             row = conn.cursor().execute(
728                 "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
1").fetchone()
729         else:
730             row = conn.cursor().execute(
731                 "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
732                 (user_id,)).fetchone()
733         if not row:
734             return None
735         d = dict(row)
736         for col in ("fusion_config", "classifier_json", "promotion_evidence"):
737             d[col] = json.loads(d[col]) if d[col] else None
738         return d
739

```

6. assistant (2026-06-18T08:05:33.541Z)

Now I have a full understanding of the `\_init\_db` method. Let me look at the test file to understand what's being tested before making changes.

7. user (2026-06-18T08:05:34.540Z)

```

1     import sqlite3
2     import pytest
3     from backend.infrastructure.database import Database
4
5     def test_database_init(tmp_path):
6         db_path = tmp_path / "test.db"
7         Database(str(db_path)) # instantiation creates the schema

```

```

8
9     conn = sqlite3.connect(str(db_path))
10    cursor = conn.cursor()
11    cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
12    tables = [r[0] for r in cursor.fetchall()]
13    assert "videos" in tables
14    assert "play_segments" in tables
15    conn.close()
16
17    def test_database_video(tmp_path):
18        db = Database(str(tmp_path / "test.db"))
19
20        vid_added = db.add_video("vid1", "vid1.mp4", "pending", [{"val": 1}])
21        assert vid_added is True
22
23        video = db.get_video("vid1")
24        assert video["id"] == "vid1"
25        assert video["filepath"] == "vid1.mp4"
26        assert video["status"] == "pending"
27        assert video["validation_results"] == [{"val": 1}]
28
29        db.clear_video_data("vid1")
30        assert db.get_video("vid1") is None
31
32    def test_database_play_segment(tmp_path):
33        db = Database(str(tmp_path / "test.db"))
34        db.add_video("vid1", "vid1.mp4", "pending", [])
35
36        sid = db.add_play_segment("vid1", 10.0, 20.0, "PlayerA", "PlayerB", {"foo": "bar"})
37        assert sid is not None
38
39        db.add_event(sid, 15.0, "smash", "PlayerA", {"speed": 300})
40
41        segments = db.query_play_segments("vid1", {"server": "PlayerA"})
42        assert len(segments) == 1
43        assert segments[0]["id"] == sid
44        assert segments[0]["start_time"] == 10.0
45        assert segments[0]["server"] == "PlayerA"
46        assert segments[0]["metadata"]["foo"] == "bar"
47
48        assert len(segments[0]["events"]) == 1
49        assert segments[0]["events"][0]["type"] == "smash"
50        assert segments[0]["events"][0]["details"]["speed"] == 300
51
52
53    # --- candidate_segments schema + behavior -----
54
55    # The schema contract: any future change to candidate_segments columns must
56    # consciously update this set, otherwise the test below fails (regression guard).
57    EXPECTED_CANDIDATE_COLUMNS = {
58        "id", "video_id", "detector", "chunk_index", "start_time", "end_time",
59        "duration", "confidence", "stats", "detection_params",
60        "confirmed_segment_id", "human_label", "notes", "created_at",
61        "user_id", # v4: personalization groundwork (nullable; NULL = local install)
62    }
63
64
65    def test_candidate_schema_contract(tmp_path):
66        Database(str(tmp_path / "test.db")) # instantiation creates the schema
67        conn = sqlite3.connect(str(tmp_path / "test.db"))
68        cols = conn.execute("PRAGMA table_info(candidate_segments)").fetchall()
69        conn.close()
70
71        names = {c[1] for c in cols}
72        assert names == EXPECTED_CANDIDATE_COLUMNS

```

```

73
74     # NOT NULL / PK flags on the load-bearing columns (cid, name, type, notnull, dflt,
pk)
75     by_name = {c[1]: c for c in cols}
76     assert by_name["id"][5] == 1 # primary key
77     for col in ("video_id", "detector", "start_time", "end_time", "duration"):
78         assert by_name[col][3] == 1, f"{col} should be NOT NULL"
79
80
81 def test_schema_version_is_current(tmp_path):
82     db_path = tmp_path / "test.db"
83     Database(str(db_path))
84     conn = sqlite3.connect(str(db_path))
85     version = conn.execute("PRAGMA user_version").fetchone()[0]
86     conn.close()
87     # v5 = per-user model store (PERSONALIZATION_PLAN.md §2). Bump with each migration.
88     assert version == 5
89
90
91 def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
92     db_path = tmp_path / "legacy.db"
93     # Simulate a pre-v1 database that only has the old tables and no version.
94     conn = sqlite3.connect(str(db_path))
95     conn.execute("CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT, status
TEXT)")
96     conn.commit()
97     conn.close()
98
99     # Opening with the new Database should add candidate_segments without error...
100    Database(str(db_path))
101    # ...and running init again must be a no-op (idempotent).
102    Database(str(db_path))
103
104    conn = sqlite3.connect(str(db_path))
105    tables = [r[0] for r in conn.execute(
106        "SELECT name FROM sqlite_master WHERE type='table'").fetchall()]
107    version = conn.execute("PRAGMA user_version").fetchone()[0]
108    conn.close()
109    assert "candidate_segments" in tables
110    assert "jobs" in tables # v2 (async job model, #16)
111    assert "user_models" in tables # v5 (per-user model store)
112    assert version == 5 # v5 = per-user model store (PERSONALIZATION_PLAN.md §2)
113
114
115 def test_candidate_round_trip(tmp_path):
116     db = Database(str(tmp_path / "test.db"))
117     db.add_video("vid1", "vid1.mp4", "processed", [])
118
119     cid = db.add_candidate_segment(
120         "vid1", "yolo_hybrid", 10.0, 25.0, chunk_index=0, confidence=0.42,
121         stats={"peak_velocity": 0.8, "track_id_count": 2},
122         detection_params={"frame_skip": 3},
123     )
124     assert cid is not None
125
126     cands = db.query_candidate_segments("vid1")
127     assert len(cands) == 1
128     c = cands[0]
129     assert c["detector"] == "yolo_hybrid"
130     assert c["start_time"] == 10.0
131     assert c["duration"] == 15.0
132     assert c["confidence"] == 0.42
133     assert c["stats"]["peak_velocity"] == 0.8
134     assert c["detection_params"]["frame_skip"] == 3
135     assert c["confirmed_segment_id"] is None

```

```

136
137     # detector filter
138     assert db.query_candidate_segments("vid1", detector="motion") == []
139     assert len(db.query_candidate_segments("vid1", detector="yolo_hybrid")) == 1
140
141
142     def test_candidate_link_and_unlabeled_filter(tmp_path):
143         db = Database(str(tmp_path / "test.db"))
144         db.add_video("vid1", "vid1.mp4", "processed", [])
145
146         cid = db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
147         sid = db.add_play_segment("vid1", 1.0, 4.0)
148
149         assert db.link_candidate_to_segment(cid, sid) is True
150         assert db.query_candidate_segments("vid1")[0]["confirmed_segment_id"] == sid
151
152         # only_unlabeled returns it while human_label is NULL
153         assert len(db.query_candidate_segments("vid1", only_unlabeled=True)) == 1
154         with db._get_connection() as conn:
155             conn.execute("UPDATE candidate_segments SET human_label = ? WHERE id = ?",
156                          ("true_rally", cid))
157             conn.commit()
158         assert db.query_candidate_segments("vid1", only_unlabeled=True) == []
159
160
161     def test_candidate_cascade_on_video_delete(tmp_path):
162         db = Database(str(tmp_path / "test.db"))
163         db.add_video("vid1", "vid1.mp4", "processed", [])
164         db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
165
166         db.clear_video_data("vid1")
167         assert db.query_candidate_segments("vid1") == []
168
169
170     # --- v4: personalization groundwork (nullable user_id; NULL = local = unchanged) -----
171
172     def _table_cols(db_path, table):
173         conn = sqlite3.connect(str(db_path))
174         cols = {c[1] for c in conn.execute(f"PRAGMA table_info({table})").fetchall()}
175         conn.close()
176         return cols
177
178
179     def test_v4_user_id_columns_present_and_nullable(tmp_path):
180         db_path = tmp_path / "test.db"
181         Database(str(db_path))
182         assert "user_id" in _table_cols(db_path, "videos")
183         assert "user_id" in _table_cols(db_path, "candidate_segments")
184
185         # The column is NULLABLE: PRAGMA table_info notnull flag must be 0.
186         conn = sqlite3.connect(str(db_path))
187         for table in ("videos", "candidate_segments"):
188             by_name = {c[1]: c for c in conn.execute(f"PRAGMA
table_info({table})").fetchall()}
189             assert by_name["user_id"][3] == 0, f"{table}.user_id must be nullable"
190             conn.close()
191
192
193     def test_v4_existing_writers_leave_user_id_null(tmp_path):
194         """NULL user_id back-compat: add_video / add_candidate_segment never set user_id, so
every
195         row a current writer produces is local (NULL) ? byte-identical to pre-v4
behaviour."""
196         db_path = tmp_path / "test.db"
197         db = Database(str(db_path))

```

```

198     db.add_video("vid1", "vid1.mp4", "processed", [])
199     db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
200
201     conn = sqlite3.connect(str(db_path))
202     vu = conn.execute("SELECT user_id FROM videos WHERE id = 'vid1']").fetchone()[0]
203     cu = conn.execute("SELECT user_id FROM candidate_segments WHERE video_id =
'vid1']").fetchone()[0]
204     conn.close()
205     assert vu is None
206     assert cu is None
207     # Existing readers are unaffected (no user_id filter in the query path).
208     assert len(db.query_candidate_segments("vid1")) == 1
209
210
211     def test_migration_v3_db_upgrades_to_v5_preserving_rows(tmp_path):
212         """A real v3 DB (jobs + candidates already populated) upgrades cleanly to v5: the
new
213         columns/tables appear, user_id back-fills to NULL, and prior rows survive the
ALTERs."""
214         db_path = tmp_path / "legacy_v3.db"
215         db = Database(str(db_path)) # fresh DB is born at the current version; simulate a
v3 state
216         # Seed pre-upgrade data, then force the version back to 3 and re-open to run v4+v5.
217         db.add_video("vidA", "a.mp4", "processed", [{"v": 1}])
218         db.add_candidate_segment("vidA", "wasb_hybrid", 2.0, 9.0, stats={"net_crossings":
3})
219         conn = sqlite3.connect(str(db_path))
220         conn.execute("DROP INDEX IF EXISTS idx_videos_user")
221         conn.execute("DROP INDEX IF EXISTS idx_candidates_user")
222         conn.execute("DROP TABLE IF EXISTS user_models")
223         # Recreate v3-shaped tables WITHOUT user_id by rebuilding (simplest: just rewind
version
224         # and drop the v4/v5 artifacts that a true v3 DB wouldn't have).
225         conn.execute("ALTER TABLE videos DROP COLUMN user_id")
226         conn.execute("ALTER TABLE candidate_segments DROP COLUMN user_id")
227         conn.execute("PRAGMA user_version = 3")
228         conn.commit()
229         conn.close()
230         assert "user_id" not in _table_cols(db_path, "videos")
231
232         # Re-open: the migration ladder must carry the v3 DB up to v5 without data loss.
233         db2 = Database(str(db_path))
234         conn = sqlite3.connect(str(db_path))
235         assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
236         assert "user_models" in {r[0] for r in conn.execute(
237             "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
238         conn.close()
239         assert "user_id" in _table_cols(db_path, "videos")
240         assert "user_id" in _table_cols(db_path, "candidate_segments")
241
242         v = db2.get_video("vidA")
243         assert v is not None and v["validation_results"] == [{"v": 1}]
244         cands = db2.query_candidate_segments("vidA")
245         assert len(cands) == 1 and cands[0]["stats"]["net_crossings"] == 3
246
247
248     def test_migration_idempotent_on_interrupted_upgrade(tmp_path):
249         """An INTERRUPTED v3/v4 upgrade must not brick the DB. A DDL ``ALTER`` auto-commits
on its own, so a crash after the ADD COLUMNS but before the ``PRAGMA user_version``
250         bump leaves the columns present while the version is stale ? re-opening re-runs the
bare ALTERs. The ladder must tolerate that (re-runnable, like the v1/v2/v5 blocks)
251         instead of raising ``OperationalError: duplicate column name``."""
252         db_path = tmp_path / "interrupted.db"
253         Database(str(db_path)) # fully migrate to the current version (v3/v4 columns all
present)

```

```

256
257 # Simulate the interruption: the v3/v4 ADD COLUMNS already landed and auto-
committed,
258 # but the process died before the version bump, so user_version is stuck at 2.
259 conn = sqlite3.connect(str(db_path))
260 conn.execute("DROP TABLE IF EXISTS user_models")          # a true v2 DB lacks the
v5 table
261 conn.execute("PRAGMA user_version = 2")
262 conn.commit()
263 conn.close()
264
265 # Re-opening re-runs v3 (jobs.policy/next_poll_at) and v4 (videos/candidate_segments
266 # .user_id) over columns that ALREADY exist ? must NOT raise, and must reach v5.
267 Database(str(db_path))
268 conn = sqlite3.connect(str(db_path))
269 assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
270 conn.close()
271 # Columns/tables are all present after the clean re-apply.
272 assert "policy" in _table_cols(db_path, "jobs")
273 assert "user_id" in _table_cols(db_path, "videos")
274 assert "user_id" in _table_cols(db_path, "candidate_segments")
275 conn = sqlite3.connect(str(db_path))
276 assert "user_models" in {r[0] for r in conn.execute(
277     "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
278 conn.close()
279
280
281 def test_add_column_idempotent_swallows_only_duplicate(tmp_path):
282     """The idempotent ADD COLUMN helper swallows ONLY 'duplicate column name'. Any other
283     OperationalError (e.g. a genuinely broken migration against a missing table) must
still
284     propagate ? the swallow is narrow, not a blanket except."""
285     db_path = tmp_path / "helper.db"
286     Database(str(db_path))          # current schema; jobs.policy already
exists
287     conn = sqlite3.connect(str(db_path))
288     cur = conn.cursor()
289     # Re-adding an existing column is tolerated (the crash-safety path).
290     Database._add_column_idempotent(cur, "ALTER TABLE jobs ADD COLUMN policy TEXT")
291     # An unrelated OperationalError (no such table) is NOT masked.
292     with pytest.raises(sqlite3.OperationalError):
293         Database._add_column_idempotent(cur, "ALTER TABLE nope ADD COLUMN x TEXT")
294     conn.close()
295
296
297 # --- v5: per-user model store CRUD -----
298
299 def test_user_models_schema_contract(tmp_path):
300     db_path = tmp_path / "test.db"
301     Database(str(db_path))
302     cols = _table_cols(db_path, "user_models")
303     assert cols == {
304         "id", "user_id", "version", "baseline_version", "feature_schema_hash",
305         "fusion_config", "classifier_json", "promotion_evidence",
306         "n_videos_at_fit", "is_active", "created_at",
307     }
308
309
310 def test_upsert_and_get_active_user_model_round_trip(tmp_path):
311     db = Database(str(tmp_path / "test.db"))
312     rid = db.upsert_user_model(
313         "user-1", version=1, baseline_version="base-2026-06",
314         feature_schema_hash="abc123",
315         fusion_config={"min_crossings": 3, "cue_flags": {"person": True}},
316         classifier_json={"model": "logreg", "w": [0.1, 0.2], "b": 0.0},

```

```

317         promotion_evidence={"promote": True, "n": 6},
318         n_videos_at_fit=6, is_active=True,
319     )
320     assert rid is not None
321
322     m = db.get_active_user_model("user-1")
323     assert m is not None
324     assert m["user_id"] == "user-1"
325     assert m["version"] == 1
326     assert m["baseline_version"] == "base-2026-06"
327     assert m["feature_schema_hash"] == "abc123"
328     assert m["fusion_config"] == {"min_crossings": 3, "cue_flags": {"person": True}}
329     assert m["classifier_json"]["w"] == [0.1, 0.2]
330     assert m["promotion_evidence"]["promote"] is True
331     assert m["n_videos_at_fit"] == 6
332     assert m["is_active"] == 1
333
334
335     def test_get_active_user_model_none_when_absent_or_inactive(tmp_path):
336         db = Database(str(tmp_path / "test.db"))
337         assert db.get_active_user_model("nobody") is None
338
339         # An inactive model is not returned as active.
340         db.upsert_user_model("user-2", version=1, is_active=False)
341         assert db.get_active_user_model("user-2") is None
342
343
344     def test_upsert_is_idempotent_on_user_version(tmp_path):
345         db = Database(str(tmp_path / "test.db"))
346         rid1 = db.upsert_user_model("user-3", version=1, n_videos_at_fit=4, is_active=True)
347         rid2 = db.upsert_user_model(
348             "user-3", version=1, n_videos_at_fit=7, is_active=True,
349             fusion_config={"min_crossings": 5},
350         )
351         # Same (user, version) ? same row updated in place, not a duplicate.
352         assert rid1 == rid2
353         m = db.get_active_user_model("user-3")
354         assert m["n_videos_at_fit"] == 7
355         assert m["fusion_config"] == {"min_crossings": 5}
356
357         conn = sqlite3.connect(str(tmp_path / "test.db"))
358         count = conn.execute(
359             "SELECT COUNT(*) FROM user_models WHERE user_id = 'user-3'").fetchone()[0]
360         conn.close()
361         assert count == 1
362
363
364     def test_activating_new_version_demotes_prior_active(tmp_path):
365         db = Database(str(tmp_path / "test.db"))
366         db.upsert_user_model("user-4", version=1, n_videos_at_fit=5, is_active=True)
367         db.upsert_user_model("user-4", version=2, n_videos_at_fit=9, is_active=True)
368
369         active = db.get_active_user_model("user-4")
370         assert active["version"] == 2 # the newer model is the single active one
371
372         # Exactly one active row for the user (the partial-unique invariant holds).
373         conn = sqlite3.connect(str(tmp_path / "test.db"))
374         n_active = conn.execute(
375             "SELECT COUNT(*) FROM user_models WHERE user_id = 'user-4' AND is_active =
376             1").fetchone()[0]
377         conn.close()
378         assert n_active == 1
379
380     def test_null_user_id_local_install_round_trip(tmp_path):

```

```

381         """NULL user_id = the local install. upsert/get must treat IS NULL correctly (not
'=NULL')."""
382         db = Database(str(tmp_path / "test.db"))
383         db.upsert_user_model(None, version=1, fusion_config={"min_crossings": 2},
384                             n_videos_at_fit=5, is_active=True)
385         m = db.get_active_user_model(None)
386         assert m is not None
387         assert m["user_id"] is None
388         assert m["fusion_config"] == {"min_crossings": 2}
389         # A named user does NOT see the local row.
390         assert db.get_active_user_model("user-x") is None
391
392
393     def test_user_model_json_columns_default_none(tmp_path):
394         """A config-only model (no classifier) stores NULL classifier_json and deserialises
to None."""
395         db = Database(str(tmp_path / "test.db"))
396         db.upsert_user_model("user-5", version=1, fusion_config={"min_crossings": 4},
397                             n_videos_at_fit=5, is_active=True)
398         m = db.get_active_user_model("user-5")
399         assert m["classifier_json"] is None
400         assert m["promotion_evidence"] is None
401         assert m["fusion_config"] == {"min_crossings": 4}
402

```

8. assistant (2026-06-18T08:05:38.423Z)

I now have full clarity. Let me perform the refactor. I'll replace the `\_init\_db` method with one that calls the extracted helpers, and add the six new helper methods.

9. user (2026-06-18T08:06:41.352Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-1/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

10. assistant (2026-06-18T08:06:44.506Z)

The refactor is complete. The SQL text, PRAGMA bumps, `\_add\_column\_idempotent` calls, ordering, and the single `with self.\_connect()` block are all preserved. Now let me run the gate. First the full test suite.

11. user (2026-06-18T08:08:49.902Z)

```

=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
obsreport (optional reporting dep): available
=====
.....SS..... [ 8%]
..... [ 16%]
..... [ 25%]
..... [ 33%]
..... [ 41%]
..... [ 50%]
..... [ 58%]
..... [ 66%]
..... [ 75%]
.....SSSSS..... [ 83%]
..... [ 91%]
..... [100%]
856 passed, 7 skipped in 119.10s (0:01:59)
=====
RESULT: PASS - all tests green

```

=====

12. assistant (2026-06-18T08:08:52.588Z)

Full suite is green: 856 passed, 7 skipped. Now the remaining three gate checks.

13. user (2026-06-18T08:10:07.846Z)

===RUFF===

All checks passed!

===MYPY===

backend\annotations\base.py:85: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 104 source files

14. assistant (2026-06-18T08:10:10.002Z)

Ruff clean, mypy success (only notes, no errors). Now the coverage gate.

15. user (2026-06-18T08:12:11.658Z)

backend\reporting\run_signals.py	14	0	100%
backend\results.py	27	0	100%
backend\sports__init__.py	17	1	94%
backend\sports\badminton.py	13	1	92%
backend\sports\base.py	12	3	75%
backend\tools__init__.py	0	0	100%
backend\tools\export.py	75	2	97%
backend\tools\rally_slicer.py	104	47	55%
backend\utils__init__.py	0	0	100%
backend\utils\geometry.py	29	0	100%
backend\utils\hashing.py	9	0	100%
backend\utils\logging_setup.py	11	0	100%
backend\utils\paths.py	27	3	89%
backend\utils\run_telemetry.py	126	12	90%
backend\utils\video.py	25	3	88%
training__init__.py	0	0	100%
training\gen0__init__.py	0	0	100%
training\gen0\harness.py	108	11	90%

TOTAL 8021 1544 81%

Required test coverage of 70% reached. Total coverage: 80.75%

856 passed, 7 skipped, 7 warnings in 115.68s (0:01:55)

=====

RESULT: PASS - all tests green

=====

16. assistant (2026-06-18T08:12:14.800Z)

All four gates are green:

- Suite: 856 passed, 7 skipped
- Ruff: All checks passed
- Mypy: Success, no issues
- Coverage: 80.75% (? 70 floor)

Let me check the database.py coverage line specifically and confirm the diff before committing.

17. user (2026-06-18T08:12:16.113Z)

```
===DIFFSTAT===
 backend/infrastructure/database.py | 349 ++++++-----
 1 file changed, 184 insertions(+), 165 deletions(-)
===STATUS===
 M backend/infrastructure/database.py
```

18. assistant (2026-06-18T08:12:18.372Z)

Only the target file is modified. Let me verify HEAD is still mine before committing, then commit, push, and open the PR.

19. user (2026-06-18T08:12:20.312Z)

```
===BRANCH===
 refactor/db-init-helpers
===HEAD===
 a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
===ONLY-MINE vs origin/master===
```

20. assistant (2026-06-18T08:12:22.684Z)

Branch is correct, HEAD is still origin/master's tip (no sibling commits crept in), and no commits ahead yet. Now I'll stage the explicit path, commit, and push.

21. user (2026-06-18T08:12:28.664Z)

```
[refactor/db-init-helpers 8c0fa48] refactor(db): extract _init_db base-table + per-version
migration helpers
 1 file changed, 184 insertions(+), 165 deletions(-)
===COMMITTED===
8c0fa48 refactor(db): extract _init_db base-table + per-version migration helpers
```

22. assistant (2026-06-18T08:12:30.687Z)

Exactly one commit ahead, all mine. Now push and open the PR.

23. user (2026-06-18T08:12:34.852Z)

```
remote:
remote: Create a pull request for 'refactor/db-init-helpers' on GitHub by visiting:
remote:   https://github.com/avidullu/badminton-highlight-indexer/pull/new/refactor/db-init-
helpers
remote:
branch 'refactor/db-init-helpers' set up to track 'origin/refactor/db-init-helpers'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      refactor/db-init-helpers -> refactor/db-init-helpers
```

24. user (2026-06-18T08:12:50.346Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/223>

25. assistant (2026-06-18T08:12:53.047Z)

PR opened at <https://github.com/avidullu/badminton-highlight-indexer/pull/223> (not merged, for owner review). All four gates were green. Returning the structured result.

26. user (2026-06-18T08:13:02.881Z)

Structured output provided successfully

27. assistant (2026-06-18T08:13:07.270Z)

Done. The refactor is complete and PR #223 is open for owner review (not merged).

Summary:

- Extracted `_create_base_tables(self, cursor)` (videos/play_segments/events CREATEs) and `_migrate_to_v1` through `_migrate_to_v5` from `_init_db` in `backend/infrastructure/database.py` as a pure verbatim cut-paste.
- `_init_db` retains its single `with self._connect()` block, the same `PRAGMA user_version` read, and the same ordered `if version < N:` guards.
- All four gates green: 856 passed / 7 skipped, ruff clean, mypy clean, coverage 80.75% (floor 70).
- Single file touched (+184 / ?165), branch cut from `origin/master`, one commit ahead, no sibling commits.